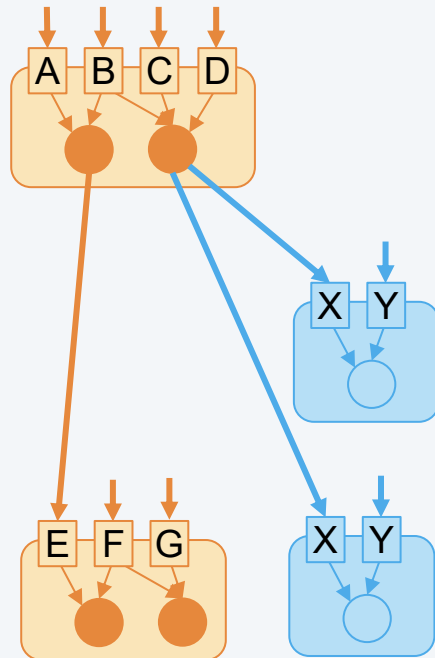




2 Klassen und Objekte

Klassen und Objekte

Methoden u. Konstruktoren

Geheimnisprinzip

Beispiel IntStack

Beispiel Color

Ein Objekt repräsentiert eine **Laufzeitkomponente** eines (objektorientierten) Programmsystems und setzt sich aus zwei Arten von Bestandteilen zusammen: **Datenkomponenten** und **Methoden**

Datenkomponenten (Attribute, *members*)

- repräsentieren die Eigenschaften eines Objekts und werden im Sinne der Datenkapselung vor der Außenwelt verborgen (*information hiding*)
- sind vor dem direkten Zugriff von außen geschützt; können nur mit entsprechenden Methoden manipuliert werden

Methoden (*methods*)

- repräsentieren Operationen, die ein Objekt auf „seinen“ Datenkomponenten ausführen kann
- werden in Form von Algorithmen realisiert
- im Zuge der Ausführung einer Methode eines Objekts können auch andere Objekte erzeugt und benutzt (indem Nachrichten an Objekte verschickt werden) werden
- unter einer **Nachricht** verstehen wir die abstrakte Form einer Methode, repräsentiert durch die Schnittstelle der Methode

Ein Objekt repräsentiert eine **Laufzeitkomponente** eines (objektorientierten) Programmsystems und setzt sich aus zwei Arten von Bestandteilen zusammen: **Datenkomponenten** und **Methoden**

Beispiel	Datenkomponenten	Methoden
Account	balance, changes	deposit, withdraw, balance,...
Stack	data, top	push, pop, isEmpty
Random	seed	intRand, realRand, rangeRand, ...
Rectangle	x, y, width, height	moveBy, contains, equals, ...
Rational	num, denom	plus, minus, divides, times, ...
Color	red, green, blue	darker, brighter, intensity, toGray, ...
String	value	length, charAt, substring, concat, ...

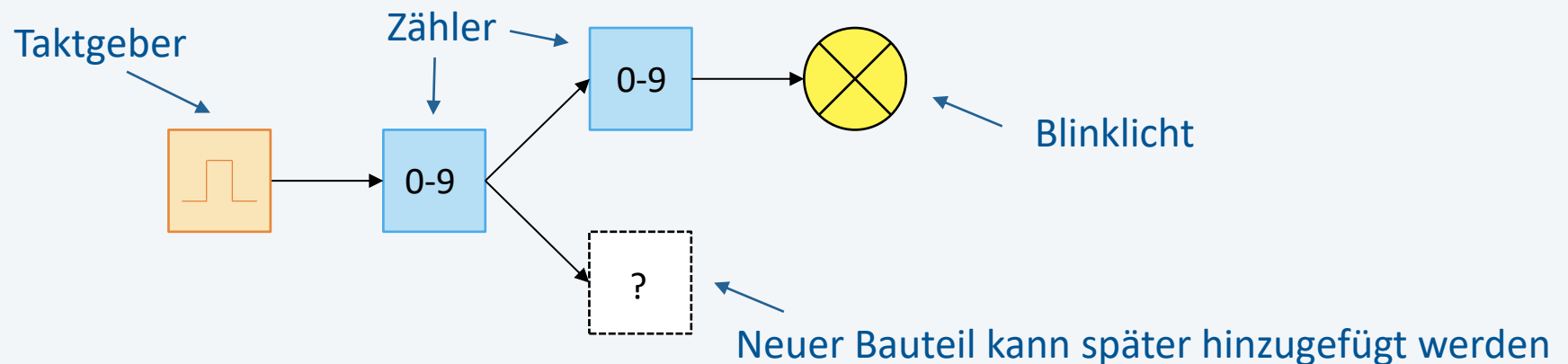
Ein Objekt ist dadurch charakterisiert, dass

- es einen **Zustand hat** (Werte der Datenkomponenten)
- bestimmte **Aufträge erledigen** kann (Nachrichten/Methoden) und
- es zu einer bestimmten **Klasse gehört**

Eine Klasse ist

- ein **benutzerdefinierter Datentyp**, der zur Bildung von Objekten, die alle über die gleichen Eigenschaften (Datenkomponenten) und Methoden verfügen, verwendet wird
- ein **abstrakter Datentyp** (ADT)
- stellt **Operationen** (Nachrichten) und **Implementierung** (Methoden) zum Zugriff auf die Datenkomponenten zur Verfügung und ermöglicht es, **beliebig viele Exemplare** (Objekte) zu erzeugen

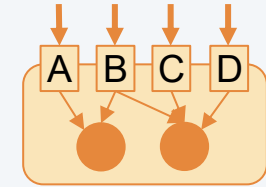
- Elemente aus der realen oder gedachten Welt (Problemwelt)
- Analogie zu technischen Bauteilen



- jedes Objekt weiß selbst, welche seiner Operationen wie auszuführen sind
- nur das Objekt selbst kennt seinen Zustand (Geheimnisprinzip)
- alle Datenobjekte (im bisherigen Sinne) können als Objekte (im objektorientierten Sinne) aufgefasst werden

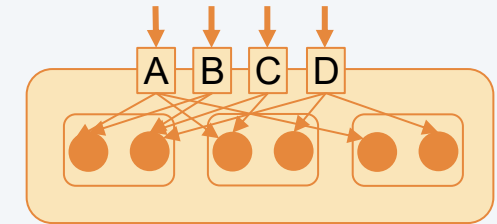
Abstrakte Datenstruktur

- ein Exemplar mit Zugriffsroutinen



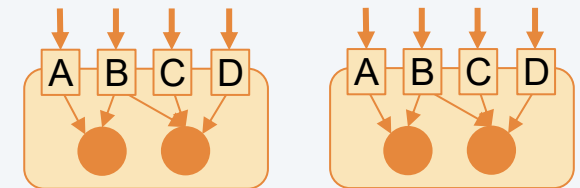
Abstrakter Datentyp

- beliebig viele Exemplare mit Zugriffsroutinen



Klasse

- beliebig viele Exemplare mit eingebauten Zugriffsroutinen (Methoden)
- Objekt enthält wie ein Verbund Datenkomponenten
- jedoch zusätzlich auch Methoden



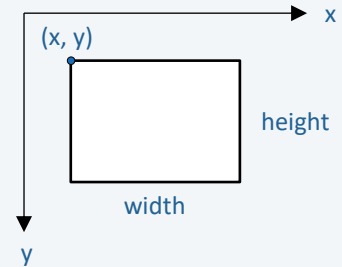
Beispiel: Klasse für Rechteck

```
class Rectangle {  
    int x, y, width, height;  
  
    void moveBy(int dx, int dy) {  
        ...  
    }  
    boolean contains(int px, int py) {  
        ...  
    }  
    void draw() {  
        ...  
    }  
}
```

Datenkomponenten
(vgl. Verbund-Datentyp)

mögliche Operationen
(Methoden)

hier noch unvollständig (siehe nächste Seite)



- Methoden sind die Algorithmen, die **ein Objekt beim Empfang von Nachrichten ausführt**, um eine Aufgabe zu erledigen
- Methoden werden **in Klassen** implementiert
- in Methoden wird auf die Datenkomponenten des Objekts zugegriffen

```
class Rectangle {  
    int x, y, width, height;
```

```
    void moveBy(int dx, int dy) {  
        x += dx;  
        y += dy;  
    }
```

x, y ... Zugriff auf Datenkomponenten

```
    boolean contains(int px, int py) {  
        return (x < px) && (px < (x + width)) && (y < py) && (py < (y + height));  
    }
```

```
    void draw() {  
        // draw rect  
    }
```

```
}
```


- Deklaration von Variablen zur Identifikation von Objekten

```
Rectangle r1, r2, r3;
```

r1:  r2:  r3: 

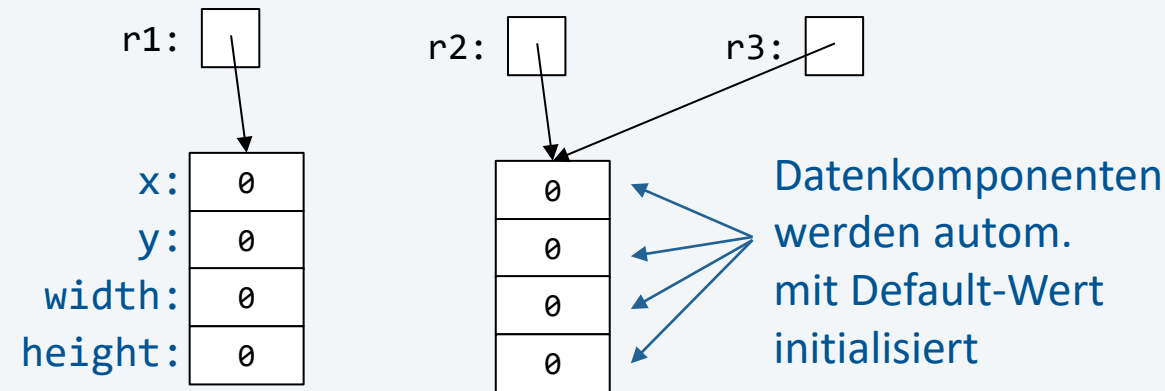
Variablen sind noch nicht initialisiert

- Erzeugung von Objekten

```
r1 = new Rectangle();
```

```
r2 = new Rectangle();
```

```
r3 = r2;
```



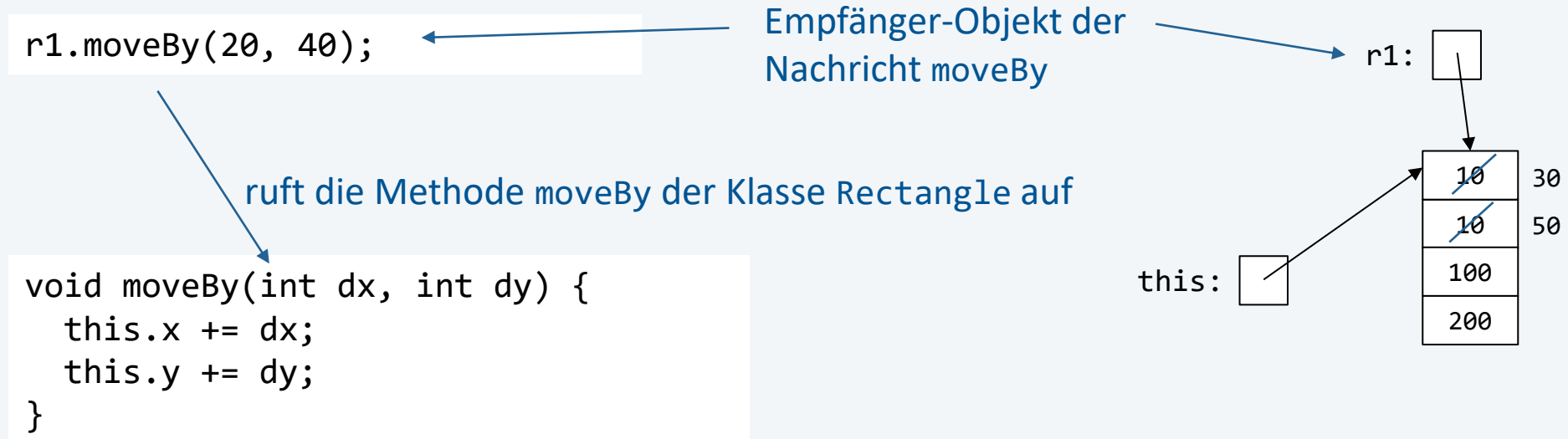
- Objekte sind dynamische Datenobjekte (liegen am Heap)
- Datenkomponenten werden automatisch mit Default-Wert initialisiert
- Objekte enthalten nur Datenkomponenten; Methoden werden nur einmal pro Klasse gespeichert

- Objekte kommunizieren über Nachrichten (*messages*) miteinander (Mechanismus, um Methoden von Objekten aufzurufen)

```
if (r1.contains(4, 3)) {  
    r1.moveBy(20, 30);  
    r1.draw();  
}
```

- Nachrichten können verschiedene Wirkung haben
 - Eigenschaft des Empfängerobjekts abfragen (z.B. `contains`)
 - Zustand des Empfängerobjekts verändern (z.B. `moveBy`)
 - Empfängerobjekt zu Aktion veranlassen (z.B. `draw`)
 - ...
- Sender der Nachricht hat keine Kenntnis, wie die Methode der Empfänger implementiert sind

Beispiel: Nachricht moveBy an ein von Variable r1 referenziertes Objekt



- innerhalb der Methode steht `this` für die Referenz auf das Empfängerobjekt

```
x += dx;  
y += dy;
```

man kann `this` weglassen, wenn sich kein Namenskonflikt ergibt

Gegenüberstellung der verwendeten Begriffe

Bisher	Im Kontext der OOP
Abstrakter Datentyp (ADT)	Klasse
Exemplar eines ADT	Objekt
Zugriffsroutine	Nachricht
Algorithmus/Prozedur	Methode
Algorithmen-/Prozeduraufruf	Senden einer Nachricht an Objekt Es gibt auch noch Algorithmenaufruf (Klassenmethoden)

- dient der Initialisierung von Objekten zum Zeitpunkt der Erzeugung
- ist eine spezifische Methode, die beim Erzeugen eines Objekts (automatisch) aufgerufen wird
- Konvention: Name des Konstruktors ist identisch mit dem der Klasse

Beispiel (Deklaration)

```
class Rectangle {  
  
    int x, y, width, height;  
  
    Rectangle(int x, int y, int width, int height) {  
        this.x = x;  
        this.y = y;  
        this.width = width;  
        this.height = height;  
    }  
    // ... moveBy, contains, draw  
}
```

Achtung: Konstruktoren haben keinen Rückgabotyp!

```
void Rectangle(...) {  
    ...  
}
```

ist eine Methode!

- Konstruktoren können überladen werden und andere Konstruktoren derselben Klasse mit `this(...)` aufrufen
- wenn kein Konstruktor vorhanden ist, erzeugt der Java-Compiler einen ohne Parameter (Standardkonstruktor)

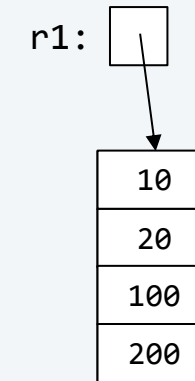
Beispiel (Konstruktoren)

```
class Rectangle {  
    int x, y, width, height;  
    Rectangle() {  
        this(0, 0, 0, 0);  
    }  
    Rectangle(int width, int height) {  
        this(0, 0, width, height);  
    }  
    Rectangle(int x, int y, int width, int height) {  
        this.x = x; this.y = y; this.width = width; this.height = height;  
    }  
    ...  
}
```

- Konstruktoren werden automatisch aufgerufen, wenn ein Objekt ihrer Klasse erzeugt wird

```
Rectangle r1 = new Rectangle(10, 20, 100, 200);
```

1. Erzeugt ein Rectangle-Objekt
2. Ruft Konstruktor auf, der die Datenkomponenten initialisiert
3. Weist Referenz auf das Objekt an r1 zu



- direkter Zugriff auf die Komponenten eines Objekts soll verhindert werden
- Zugriffsrechte werden z.B. durch `private` und `public` definiert

```
public class Rectangle {  
    private int x, y, width, height;  
  
    public Rectangle(int x, int y, int width, int height) { ... }  
    public void moveBy(int dx, int dy) { ... }  
    public boolean contains(int px, int py) { ... }  
    public void draw() { ... }  
}
```

`private` ... nur innerhalb der
Klasse sichtbar

`public` ... global sichtbar

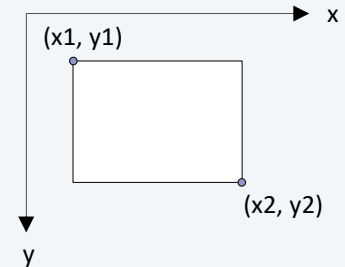
- Verwendung

```
Rectangle r1 = new Rectangle(10, 20, 100, 200);  
r1.width = 42;  
System.out.print(r1.x);
```

wegen `private` nicht möglich!

Zugriffsmethoden zu privaten Datenkomponenten (hier nur für *x* und *width*)

```
public class Rectangle {  
    private int x, y, width, height;  
    ...  
    public int getX() { ← nur Lesezugriff für Eigenschaft x  
        return x;  
    }  
    public int getWidth() { ← Lese- und Schreibzugriff für  
        return width;      Eigenschaft width  
    }  
    public void setWidth(int value) {  
        width = (value > 0) ? value : 0;  
    }  
}
```



Konsequenzen

- Implementierung der Datenkomponenten kann ausgetauscht werden

```
int x1, y1, x2, y2;
```

- Zusatzfunktionen (z.B. Vorbedingungen prüfen) beim Zugriff möglich

Beispiel

```
public class Point {  
  
    private double x = 1.0;  
    private double y = 1.0;  
  
    {  
        x = 2.0; y = 2.0;  
    }  
  
    public Point() {  
        this.x = 3.0;  
        this.y = 3.0;  
    }  
  
}
```

```
Point p = new Point();
```

Initialisierungsreihenfolge:

Zeitpunkt	x	y
1. Default-Wert	0.0	0.0
2. Deklaration der Datenkomponente	1.0	1.0
3. Init-Block (unüblich)	2.0	2.0
4. Konstruktor	3.0	3.0

guter Stil: Default-Wert oder Konstruktor

```
public class Point {  
  
    private double x;  
    private double y;  
  
    public Point() {  
        this.x = 1.0;  
        this.y = 1.0;  
    }  
  
}
```

Beispiel: Gesucht ist eine Klasse `IntStack` für die Verwaltung von Integer-Objekten nach dem LIFO-Prinzip

Schritt 1: Welche Operationen werden benötigt?

```
IntStack s = new IntStack(32);  
int n = 1_000_000_000;  
while (n > 0) {  
    s.push(n % 2);  
    n = n / 2;  
}  
while (!s.isEmpty()) {  
    System.out.print(s.pop());  
}
```

111011100110101100101000000000

Schritt 2: Welche Datenkomponenten werden für die Implementierung benötigt?

```
int[] stack;  
int top;
```

Schritt 3: Klassendeklaration

```
public class IntStack {  
  
    private int[] stack;  
    private int top;  
  
    public IntStack(int capacity) {  
        assert capacity >= 0 : "illegal capacity";  
        this.stack = new int[capacity];  
        this.top = -1;  
    }  
  
    public void push(int e) {  
        assert !isFull() : "stack must not be full";  
        top++;  
        stack[top] = e;  
    }  
    ...  
}
```

Datenobjekte sind nicht von außen zugänglich

Konstruktor

Methoden

oder:
`stack[++top] = e;`

Schritt 3: Klassendeklaration

```
public class IntStack {  
    ...  
    public int pop() {  
        assert !isEmpty() : " stack must not be empty";  
        int e = stack[top];  
        top--;  
        return e;  
    }  
  
    public boolean isEmpty() {  
        return top == -1;  
    }  
  
    public boolean isFull() {  
        return top == stack.length - 1;  
    }  
}
```

} oder:
return stack[top--];

- eine Farbe ist ein durch das Auge/Gehirn vermittelter Sinneseindruck, der durch elektromagnetische Strahlung hervorgerufen wird
- gesucht ist eine Klasse Color zur Repräsentation von Farben

```
public class Color {  
  
    private int red;  
    private int green;  
    private int blue;  
  
    public Color(int red, int green, int blue) {  
        this.red = red;  
        this.green = green;  
        this.blue = blue;  
    }  
    public int getRed() { return red; }  
    public int getGreen() { return green; }  
    public int getBlue() { return blue; }  
    ...  
}
```

Datenkomponenten:

Intensität für Rot-, Grün-, Blauanteil (0-255)

R	255	0	0	255	0	51	51
G	0	255	0	255	0	51	102
B	0	0	255	255	0	153	0

Farbe       

weitere Methoden auf den nächsten Seiten

Methode darker

```
public class Color {  
    ...  
    public void darker() {  
        double f = 0.95;  
        red = (int)(red * f);  
        green = (int)(green * f);  
        blue = (int)(blue * f);  
    }  
    ...  
}
```

Benutzung:

```
Color c = new Color(0, 255, 0);  
for (int i = 0; i < 10; i++) {  
    int x = 8 + 8 * i;  
    StdDraw.setPenColor(c.red(), c.green(), c.blue());  
    StdDraw.filledRectangle(x, 50, 3, 3);  
    c.darker();  
}
```



Methode equals

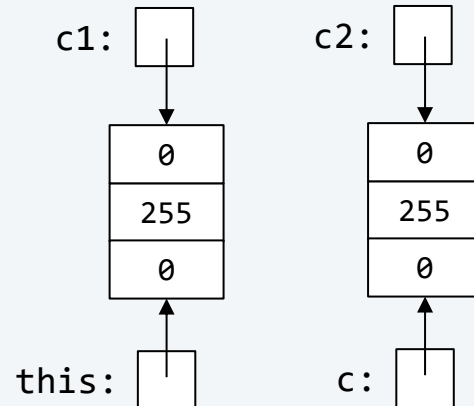
in Methoden der Klasse Color ist der Zugriff erlaubt!

```
public class Color {  
    ...  
    public boolean equals(Color c) {  
        return red == c.red && green == c.green && blue == c.blue;  
    }  
}
```

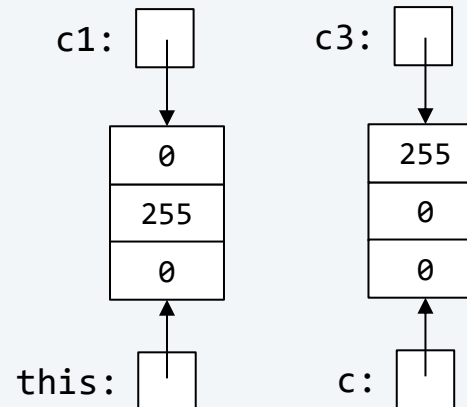
Benutzung:

```
Color c1 = new Color(0, 255, 0);  
Color c2 = new Color(0, 255, 0);  
Color c3 = new Color(255, 0, 0);
```

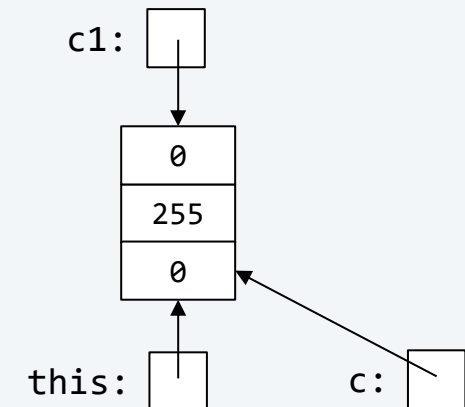
c1.equals(c2)



c1.equals(c3)



c1.equals(c1)



- Die Luminanz ist ein Maß für die Helligkeit von Bildpunkten
- NTSC-Standardformel für Helligkeit: $0.299r + 0.587g + 0.114b$

Beispiele:

R	255	0	0	255	0	51	51
G	0	255	0	255	0	51	102
B	0	0	255	255	0	153	0
Farbe							
Lum.	76	150	29	255	0	62	75

```
public class Color {  
    ...  
    public double intensity() {  
        return (0.299 * red) + (0.587 * green) + (0.114 * blue);  
    }  
    ...  
}
```

Aufgabe: Konvertierung einer Farbe in entsprechende Graustufe

- Hinweis: wenn alle drei R/G/B-Komponenten den gleichen Wert haben, ergibt dies eine Graustufe zwischen 0 (schwarz) und 255 (weiß)
- Welchen Wert soll man für eine bestimmte Farbe wählen? Den Luminanz-Wert

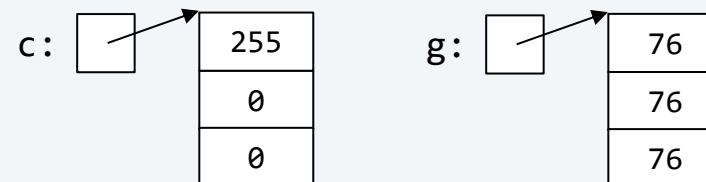
```
public class Color {  
    ...  
    public Color toGray() {  
        int i = (int)this.intensity();  
        return new Color(i, i, i);  
    }  
    ...  
}
```

Beispiele:

Farbe							
Lum.	76	150	29	255	0	62	75
Graustufe							

■ Benutzung:

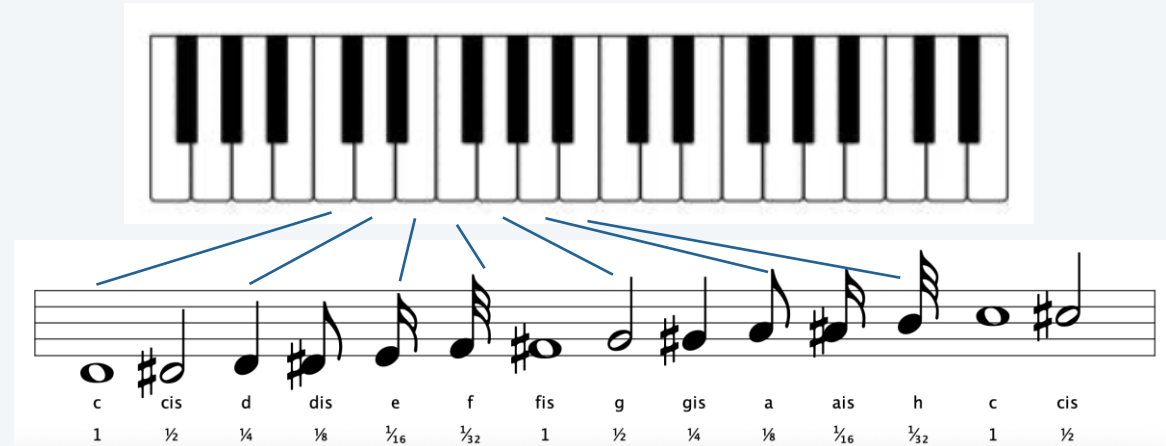
```
Color c = new Color(255, 0, 0);  
Color g = c.toGray();
```



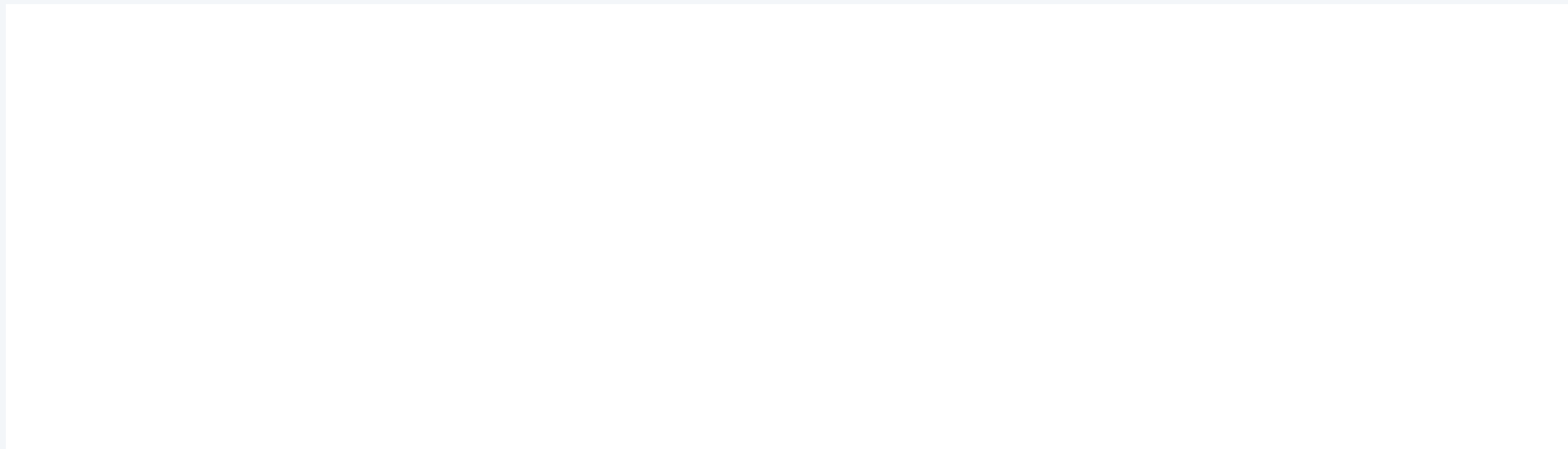
Aufgabe: Klasse für Musiknote

Eigenschaften

- Notennamen: c, d, e, f, g, a, h
- Halbtöne: cis, dis, fis, gis, ais
- Notenwerte: 1, $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{16}$, $\frac{1}{32}$



Verwendung: Befülle Feld mit Noten-Objekten (siehe Abbildung)



Methode increment()



	tone	sharp
c	Tone.C	false
cis	Tone.C	true
d	Tone.D	false
dis	Tone.D	true
e	Tone.E	false
f	Tone.F	false
fis	Tone.F	true
g	Tone.G	false
gis	Tone.G	true
a	Tone.A	false
ais	Tone.A	true
h	Tone.H	false



	tone	sharp
cis	Tone.C	true
d	Tone.D	false
dis	Tone.D	true
e	Tone.E	false
f	Tone.F	false
fis	Tone.F	true
g	Tone.G	false
gis	Tone.G	true
a	Tone.A	false
ais	Tone.A	true
h	Tone.H	false
c	Tone.C	false
octave++		